

EECS 841 Homework 1

JOHNPAUL MBAGWU

1

Part I

(a) Finding the 1D index of a pixel in row-major order

Given: An image I have 120 columns and 100 rows. It is stored in row-major order, meaning rows are stored sequentially in memory. The white pixel is located at $(x = 38, y = 52)$.

let k be the number of columns

Row-major order formula for the 1D index:

$$\text{index (I)} = y \times k + x$$

Let's substitute the given values:

$$\text{index (I)} = 52 \times 120 + 38$$

$$\text{index (I)} = 6240 + 38$$

$$\text{index (I)} = 6278$$

Hence, the white pixel (intensity 255) is located in the 1D array at index 6278.

(b)

Here, let C be column and R as Row

	C 0	C 1	C 2	C 3	C 4
R 0	1	2	3	4	5
R 1	6	7	8	9	10
R 2	11	12	13	14	15
R 3	16	17	18	19	20

- $(1, 0)$: $y = 0, x = 1$. Hence, column 1 is the first row (because y begins at 0). Observing the first row: 1, 2, 3, 4, 5. Consequently, column indices 0 – 4. Thus, $x = 1$ equals 2. Every cell in the first row, $|1|2|3|4|5|$, is a column. Thus, the first row is $y = 0$. Intensity is 2 since $x = 0$ is 1.
- $(3, 1)$: $y = 1, x = 3$. The second row ($y = 1$) is 6, 7, 8, 9, 10. The fourth element, which is 9, is $x = 3$.
- $(2, 2)$: $y = 2, x = 2$. Row three ($y = 2$): 11, 12, 13, 14, 15. 13 is $x = 2$.

The locations of the Pixel Intensity Values $I(1,0)$, $I(3,1)$, and $I(2,2)$ are:

$I(1,0)$: Col 1, Row 0 $\rightarrow$ 2

$I(3,1)$: Col 3, Row 1 $\rightarrow$ 9

$I(2,2)$: Col 2, Row 2 $\rightarrow$ 13

Thus, the pixel intensities are 2, 9, and 13.

Type of image: image in grayscale. The gray shades, neither binary (0/255) nor color (RGB), are represented by the intensity values (1–20). Since binary images only have two possible values—0 and 1—it is not a binary image. As a result, it is merely a representation of pixel values, with each pixel location being given an arbitrary integer.

Image brightness: Pixel values in the image range from 1 to 20. These are comparatively low values. As a result, we might say that the image is generally dark. We could classify it as bright if the values were in a higher range.

Part II

Python Code

The following Python code defines the function `flipflopimage`, which takes an image of size $m \times n$ (where m and n can be either even or odd) stored as a matrix. The function displays the original image, rearranges its quadrants as required, displays the modified image, and returns the rearranged image as a matrix of the same size as the original.

```
from PIL import Image
import numpy as np

def flipflopimage(image_path):
    """
    accepts an image of size m x n (where m and n can be even or odd), stores

    Rearrangement: The "flip-flop" operation essentially rotates the image 180
                    degrees. This can be achieved by swapping the quadrants. Specifically:

                    - The top-left quadrant moves to the bottom-right quadrant
                    - the top-right quadrant moves to the bottom-left quadrant
                    - the bottom-left quadrant moves to the top-right quadrant
                    - and the bottom-right comes to the top-left quadrant.

    Args:
        image_path (str): The path to the image file.

    Returns:
        numpy.ndarray: The rearranged image as a NumPy array.
    """
```

```

try:
    # 1. Let's read the Image as:
    img = Image.open(image_path)
    img_matrix = np.array(img)
    m, n = img_matrix.shape[:2] # Handle color images (m x n x 3) or gray

    # 2. Let's display the Original Image
    print("Original_Image:")
    original_img = Image.fromarray(img_matrix) # Create image from array
    original_img.show()

    # 3. Let's calculate the Quadrant Sizes as:
    m_mid = m // 2
    n_mid = n // 2

    # 4. Extract Quadrants
    q1 = img_matrix[:m_mid, :n_mid] # Top-left
    q2 = img_matrix[:m_mid, n_mid:] # Top-right
    q3 = img_matrix[m_mid:, :n_mid] # Bottom-left
    q4 = img_matrix[m_mid:, n_mid:] # Bottom-right

    # 5. Let's create a New Matrix for the Rearranged Image
    rearranged_img_matrix = np.zeros_like(img_matrix) # Let's create a ma

    # 6. Let's place Quadrants in New Positions
    rearranged_img_matrix[:m_mid, :n_mid] = q4 # Top-left becomes Bot
    rearranged_img_matrix[:m_mid, n_mid:] = q3 # Top-right becomes Bo
    rearranged_img_matrix[m_mid:, :n_mid] = q2 # Bottom-left becomes
    rearranged_img_matrix[m_mid:, n_mid:] = q1 # Bottom-right becomes

    # 7. Let's display the Rearranged Image
    print("\nRearranged_Image_(Flip-Flop):")
    rearranged_img = Image.fromarray(rearranged_img_matrix)
    rearranged_img.show()

    return rearranged_img_matrix

except FileNotFoundError:
    print(f"Error:_Image_file_not_found_at_{image_path}")
    return None
except Exception as e:
    print(f"An_error_occurred:{e}")
    return None

if __name__ == '__main__':

```

```
# Let's use the example from the HW (replace with your image path)
image_file = "image.png" # You can replace with the correct filename and/

rearranged_image = flipflopimage(image_file)

if rearranged_image is not None:
    print("\nRearranged_image_matrix:")
    print(rearranged_image)
```